# Apuntes PyTorch: guía de parámetros

Referencia por **función / clase**. Para cada parámetro: qué es y qué hay que meter, qué pasa si **se omite** (valor por defecto), y qué pasa si se pone **otro valor** o se hace mal (significado o error). Los mensajes de error pueden variar un poco según la versión de PyTorch.

Convención de formas: **B** = nº de muestras (batch), **F** = nº de features, **C** = nº de clases.

---

## 1. Crear tensores

**`torch.tensor(data, dtype=None, device=None, requires_grad=False)`**

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `data` | Lista, lista de listas, tupla, array NumPy o número. **Obligatorio.** | `TypeError` | Lista irregular `[[1,2],[3]]` → `ValueError: expected sequence of length 2 at dim 1 (got 1)`. Con `[1,"a"]` → error de tipo. |
| `dtype` | Tipo de dato: `torch.float32` (= `torch.float`), `torch.float64` (= `torch.double`), `torch.long` (= `torch.int64`), `torch.int32`, `torch.bool`. | Se **deduce**: enteros → `int64`; decimales → `float32`; array NumPy float64 → `float64`; `True/False` → `bool`. | Un `int64` **no entra** en `nn.Linear` (`mat1 and mat2 must have the same dtype`). Etiquetas de `CrossEntropyLoss` deben ser `long`. Pasar un texto (`dtype="float"`) → `TypeError: argument 'dtype' must be torch.dtype, not str`. |
| `device` | `"cpu"`, `"cuda"`, `"cuda:0"`… dónde vive el tensor. | CPU. | `"cuda"` sin GPU → `AssertionError/RuntimeError: Torch not compiled with CUDA enabled` o `No CUDA GPUs available`. Mezclar tensores cpu y cuda en una operación → `Expected all tensors to be on the same device`. |
| `requires_grad` | `True` si quieres derivar respecto a este tensor (los parámetros de las capas ya lo tienen a `True`). | `False`. | `True` sobre un tensor entero → `RuntimeError: Only Tensors of floating point and complex dtype can require gradients`. |

**`torch.randn(*size, dtype=None, device=None, requires_grad=False)`**

Números aleatorios de una Normal(0,1).

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `*size` | Dimensiones: `randn(2,3,4)` o `randn((2,3,4))`. La 1ª dimensión es el nº de muestras. | Sin tamaños → `TypeError: randn() received an invalid combination of arguments`. | Dimensión negativa → `RuntimeError: Trying to create tensor with negative dimension`. Decimales (`randn(2.5)`) → `TypeError`. |

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `dtype`, `device`, `requires_grad` | Igual que en `torch.tensor`. | `float32`, CPU, `False`. | `dtype` entero → `RuntimeError: normal_kernel_cpu not implemented for 'Long'`. |

Variante `torch.rand(...)`: uniforme en [0,1). `torch.zeros/ones(...)`: ceros / unos.

**`torch.randperm(n)`**

Permutación aleatoria de `0..n-1`. `n` entero **obligatorio**. Se usa para barajar índices (`idx[:120]` = 120 índices aleatorios). `n` decimal → `TypeError`. `n` negativo → `RuntimeError`. Sin semilla cambia en cada ejecución (en `training_iris.py` los splits no son reproducibles).

**`torch.manual_seed(seed)`**

Fija la semilla del generador de PyTorch → mismos números aleatorios (pesos iniciales, `randn`, shuffle). `seed` entero. Sin llamarla, cada ejecución da resultados distintos. No afecta a NumPy (hay que añadir `np.random.seed(...)`) ni a `train_test_split` (usa su propio `random_state`).

---

## 2. Operaciones y métodos sobre tensores

| Función | Parámetros | Si se omite | Otros valores / errores |
|---|---|---|---|
| `t.size(dim=None)` / `t.shape` | `dim`: índice de dimensión. | Devuelve la forma completa. | `t.size(5)` en un tensor 2D → `IndexError: Dimension out of range (expected to be in range of [-2, 1], but got 5)`. Admite negativos (`-1` = última). |
| `torch.flatten(input, start_dim=0, end_dim=-1)` | `start_dim`: primera dim a aplanar; `end_dim`: última. | `start_dim=0` aplana **todo, incluido el batch** → vector 1D. | Para (2,3,4): `start_dim=1` → (2,12) ✔ (lo que se quiere antes de un `Linear`). `start_dim=0` → (24,) ✘ (pierdes las muestras). Fuera de rango → `IndexError`. |
| `torch.sum(input, dim=None, keepdim=False)` | `dim`: eje que se suma. | Suma **todos** los elementos → un escalar. | `dim=1` suma por filas (una suma por muestra); `dim=0` por columnas. `keepdim=True` conserva ese eje con tamaño 1. |
| `torch.argmax(input, dim=None, keepdim=False)` | `dim`: eje donde buscar el máximo. | Aplana todo y devuelve **un solo índice** (bug silencioso en clasificación). | `dim=1` con forma (B,C) → clase predicha por muestra (B,). `dim=0` → qué muestra tiene mayor valor por clase (casi nunca lo que quieres). |
| `torch.round(input, decimals=0)` | `decimals`: decimales a conservar. | Redondea a entero. | `decimals=2` → 2 decimales; negativos redondean a decenas, centenas... |
| `torch.cat(tensors, dim=0)` | Lista/tupla de tensores y eje de unión. | Une por filas (`dim=0`). | Tamaños incompatibles en el resto de ejes → `RuntimeError: Sizes of tensors must match except in dimension 0`. Lista vacía → error. |
| `t.item()` | Sin parámetros: convierte tensor de **1 elemento** a número Python. | — | Con más de 1 elemento → `ValueError/RuntimeError: a Tensor with N elements cannot be converted to Scalar`. Se usa `loss.item()` para acumular sin retener el grafo. |

| Función | Parámetros | Si se omite | Otros valores / errores |
|---|---|---|---|
| `t.float()`, `t.long()`, `t.to(...)` | Cambian dtype/device y devuelven una copia. | — | `t.to("cuda")` sin GPU → error. No modifican `t` (hay que reasignar). |
| `t.numpy()` | Tensor CPU → array NumPy (**comparten memoria**). | — | Si `t.requires_grad` → `RuntimeError: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy()`. Si está en GPU → `TypeError: can't convert cuda:0 device type tensor to numpy` (usar `.cpu()`). |
| `t.detach()` | Corta el tensor del grafo de autograd (sin gradiente). | — | `with torch.no_grad():` tiene el mismo efecto para todo un bloque. |

---

## 3. `nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)`

Capa densa: `y = x·Wᵀ + b`. Pesos `W` de forma `(out, in)`. **Parámetros entrenables = in·out + out** (sin bias: in·out).

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `in_features` | Entero = tamaño de la **última dimensión** de la entrada (F). Para entrada (B,F) → `F`. Tras `flatten`, el producto de las dims. | **Obligatorio**: `TypeError: missing 1 required positional argument`. | Si no coincide con la entrada → `RuntimeError: mat1 and mat2 shapes cannot be multiplied (6x4 and 12x2)`. Decimal o texto → `TypeError`. |
| `out_features` | Nº de neuronas de la capa = tamaño de la salida. En la última capa: `C` (clasificación) o `1` (regresión). | **Obligatorio**. | En clasificación, `out_features` ≠ nº de clases → no hay error de forma, pero `Target N is out of bounds` si hay etiquetas ≥ `out_features`; si es mayor, hay clases "fantasma" que nunca se usan. |
| `bias` | `True/False`: si suma un vector `b`. | `True`. | `False`: se pierden `out` parámetros y la capa pasa por el origen. Se usa antes de `BatchNorm` (que ya tiene su desplazamiento). |
| `device`, `dtype` | Dónde y con qué tipo se crean los pesos. | CPU, `float32`. | Con `dtype=torch.float64` el input debe ser `float64` también. |

**Entrada admitida:** tensor `float`, con forma `(B,F)` o `(…,F)`. - `(2,3,4)` en `Linear(4,·)` **funciona** (se aplica a la última dim → salida (2,3,·)); en `Linear(12,·)` falla → hay que hacer `flatten(start_dim=1)`. - Entrada 1D `(F,)` funciona (una sola muestra, salida `(out,)`). - Entero → error de dtype; NumPy → `TypeError: argument 'input' must be Tensor, not numpy.ndarray`.

---

## 4. Funciones de activación

| Clase y parámetros | Qué hace y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `nn.ReLU(inplace=False)` | max(0,x). `inplace` = bool. | `False`: devuelve un tensor nuevo. | `True` modifica el tensor de entrada (ahorra memoria) pero puede dar `RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation`. |
| `nn.Sigmoid()` / `nn.Tanh()` | Sin parámetros. Sigmoid → (0,1); Tanh → (−1,1). | — | Se satura con valores grandes (gradiente ~0). Meter parámetros → `TypeError`. |
| `nn.LeakyReLU(negative_slope=0.01)` | ReLU con pendiente pequeña para x<0. | 0.01. | `negative_slope=0` equivale a ReLU. |
| `nn.PReLU(num_parameters=1, init=0.25)` | Como LeakyReLU pero la pendiente es **entrenable**. `num_parameters`: `1` (una pendiente para todos) o `C` (una por canal/feature, debe coincidir con la dim 1 de la entrada). `init`: valor inicial. | 1 parámetro, pendiente 0.25. | **Añade `num_parameters` parámetros entrenables** (por eso el nº de parámetros del modelo sube). Si `num_parameters` ≠ nº de features → `RuntimeError`. |
| `nn.Softmax(dim=None)` | Convierte logits en probabilidades (≥0, suman 1 a lo largo de `dim`). | Se elige `dim` implícitamente y sale un `UserWarning` (deprecado). | Forma (B,C): `dim=1` o `-1` ✔ suma 1 por muestra. `dim=0` ✘ sin error pero cada **columna** suma 1. `dim=2` en un tensor 2D → `IndexError: Dimension out of range`. |

Otras en `torch.nn`: `ELU`, `GELU`, `SiLU`, `Softplus`, `Hardtanh`, `LogSoftmax(dim)`.

---

## 5. Construir modelos

**`nn.Sequential(*capas)`**

| Parámetro | Qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `*capas` | **Instancias** de módulos separadas por comas, en el orden de ejecución: `nn.Sequential(nn.Linear(4,10), nn.ReLU(), nn.Linear(10,3))`. | `Sequential()` vacío: no hace nada (devuelve la entrada). | Poner la clase sin paréntesis (`nn.ReLU`) → `TypeError: ... is not a Module subclass`. Pasar una lista (`nn.Sequential([..])`) → el mismo `TypeError` (usar `*lista`). Capas con tamaños incompatibles → error de shapes al hacer la inferencia, **no al crear** el modelo. |

Se puede indexar `model[0]`, `model[0].weight`, `model[0].bias`, `len(model)`.

`torchinfo.summary(model, input_size=None, input_data=None, batch_dim=None, col_names=None, depth=3, device=None, verbose=1)`

| Parámetro | Qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `model` | El `nn.Module`. | Obligatorio. | — |
| `input_size` | Tupla **incluyendo el batch**: `(1,4)` = 1 muestra de 4 features. | Sin él solo muestra capas y parámetros (sin tamaños de salida). | `(4,)` se interpreta como batch=4 sin features → error de shapes. Tamaño que no encaja → `RuntimeError: Failed to run torchinfo...` con el error original. |
| `input_data` | Tensor real en vez de `input_size`. | — | Debe ser `float` para un `Linear`. |
| `col_names` | Tupla de columnas: `"input_size"`, `"output_size"`, `"num_params"`, `"mult_adds"`, `"trainable"`. | `("output_size","num_params")` si hay `input_size`. | Nombre inexistente → `ValueError`. |
| `depth` | Niveles de anidamiento que se despliegan. | 3. | `depth=1` solo capas de primer nivel. |
| `verbose` | `0` nada, `1` imprime, `2` detalle. | 1. | `0` solo devuelve el objeto (no imprime). |

### Clase `MLP(n_in, n_out, hidden=(64,64,32), p_drop=0.0)` (definida en `tabular_data`)

| Parámetro | Qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `n_in` | Nº de features de entrada (F). | Obligatorio. | Distinto de las columnas reales → error de shapes en la primera inferencia. |
| `n_out` | `1` regresión; `C` clasificación. | Obligatorio. | Menos que el nº de clases → `Target N is out of bounds`. |
| `hidden` | **Tupla** con el nº de neuronas de cada capa oculta. | `(64,64,32)`: 3 capas ocultas. | `hidden=(64)` es un `int`, no tupla → `TypeError: 'int' object is not iterable`; escribir `(64,)`. `hidden=()` → una sola `Linear` (modelo lineal). |
| `p_drop` | Probabilidad de Dropout tras cada ReLU (0–1). | `0.0` → no se añade Dropout. | `>0` añade `nn.Dropout(p)`; `>1` o `<0` → `ValueError` al crearlo. |

---

## 6. Clasificación: logits, probabilidades y clases

| Función | Parámetros | Si se omite | Otros valores / errores |
|---|---|---|---|
| `torch.distributions.Categorical(probs=None, logits=None)` | **Exactamente uno** de los dos: `probs` (filas ≥0 que suman 1) o `logits` (sin normalizar). | Ninguno → `ValueError: Either probs or logits must be specified`. | Los dos a la vez → `ValueError: Either probs or logits must be specified, but not both`. Probabilidades que no suman 1 → `ValueError` de validación. |
| `dist.sample(sample_shape=())` | Nº de muestras extra. | Una muestra por fila. | Aleatorio: distinto en cada llamada (a diferencia de `argmax`). |

Patrón: `logits = model(x)` → `probs = softmax(logits, dim=1)` → `argmax(dim=1)`. Para predecir la clase **no hace falta** el softmax (el máximo es el mismo).

**¿Sigmoid o Softmax al final?** Softmax para clases excluyentes (Iris). Con `CrossEntropyLoss` **no** se añade ninguna: la loss ya incluye `LogSoftmax`. Si la añades no hay error, pero entrena peor.

---

## 7. Funciones de pérdida

**`nn.CrossEntropyLoss(weight=None, ignore_index=-100, reduction='mean', label_smoothing=0.0)`**

Se llama `criterion(prediccion, objetivo)`: **pred** = logits float `(B,C)`; **objetivo** = índices `long (B,)` con valores `0..C-1`.

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `weight` | Tensor **float de longitud C** (un peso por clase), en el mismo device que el modelo. Típico: `N/(C·cnt_c)`. | `None`: todas las clases pesan igual. | Longitud ≠ C → `RuntimeError`. Tensor `long` o en otro device → error de dtype/device. Pesos muy extremos (≈99 para una clase con 5 muestras) desestabilizan el entrenamiento. Con `'mean'` divide por la suma de pesos de las muestras del batch. |
| `ignore_index` | Etiqueta que se ignora en la loss. | `-100`. | Útil para relleno; una etiqueta `-100` en el target no da error. |
| `reduction` | `'mean'`, `'sum'` o `'none'`. | `'mean'` (un escalar). | `'none'` devuelve un vector de B pérdidas y `.backward()` falla: `grad can be implicitly created only for scalar outputs`. Otro texto → `ValueError`. |
| `label_smoothing` | Número en [0,1] que suaviza las etiquetas. | `0.0`. | Fuera de [0,1] → `RuntimeError/ValueError`. |

**Errores de uso:** - Argumentos al revés (`criterion(y, y_pred)`) → error de dtype/forma. - Target `float` → `expected scalar type Long but found Float`. - Etiquetas 3..9 con 7 salidas → `IndexError: Target 9 is out of bounds` (hay que restar el mínimo). - Target de forma `(B,1)` → error de forma (usar `y.squeeze()` o `y.view(-1)`). - Meter softmax antes de la loss → no hay error, resultados peores.

**`nn.MSELoss(reduction='mean')`**

Error cuadrático medio, para regresión. `pred` y `target` **float y con la misma forma**. - `reduction`: `'mean'` por defecto; `'sum'`; `'none'`. - `pred (B,1)` y `target (B,)` → **no da error**: hace broadcasting a (B,B) y calcula una loss equivocada (solo un `UserWarning: Using a target size that is different to the input size`). Por eso en tabular se usa `y.reshape(-1,1)`. - Target `long` → `RuntimeError: Found dtype Long but expected Float`.

---

# 8. Optimizadores y paso de entrenamiento

**`optim.SGD(params, lr=1e-3, momentum=0, dampening=0, weight_decay=0, nesterov=False)`**

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `params` | `model.parameters()` (o lista de dicts por grupos con distinto lr). **Obligatorio.** | `TypeError`. | Pasar `model` en vez de `model.parameters()` → `TypeError: optimizer can only optimize Tensors, but one of the params is Module`. Sin parámetros → `ValueError: optimizer got an empty parameter list`. |
| `lr` | Tasa de aprendizaje (típico 0.1–0.001 en SGD). | `1e-3` en versiones recientes (antes era obligatorio). | Negativo → `ValueError: Invalid learning rate`. Muy alto → la loss sube o `nan` (sin error). Muy bajo → aprende muy lento. |
| `momentum` | 0–1 (típico 0.9): acumula la dirección de pasos previos. | `0`: SGD puro. | Negativo → `ValueError: Invalid momentum value`. |
| `dampening` | Amortigua el momento. | `0`. | — |
| `weight_decay` | λ de la regularización **L2**. | `0`: sin regularización. | Negativo → `ValueError`. Muy alto → los pesos tienden a 0 (underfitting). |
| `nesterov` | Bool: momento de Nesterov. | `False`. | `True` exige `momentum>0` y `dampening=0`, si no `ValueError: Nesterov momentum requires a momentum and zero dampening`. |

**`optim.Adam(params, lr=1e-3, betas=(0.9,0.999), eps=1e-8, weight_decay=0, amsgrad=False)`**

- `lr`: por defecto 1e-3 (con 0.1, como en el notebook de optimizadores, suele ser inestable).
- `betas`: tupla de dos números en [0,1); fuera de rango → `ValueError: Invalid beta parameter at index 0`.
- `eps`: estabilidad numérica; negativo → `ValueError`.
- `weight_decay`, `amsgrad`: como arriba. `Adam` adapta el paso por parámetro, por eso suele converger más rápido que SGD sin ajustar.

**`optim.RMSprop(params, lr=1e-2, alpha=0.99, eps=1e-8, weight_decay=0, momentum=0, centered=False)`**

- `alpha`: suavizado de la media de gradientes² (en [0,1); otro valor → `ValueError`).
- `centered=True`: normaliza con la varianza estimada.
- El resto igual que en SGD.

### Métodos del bucle

| Llamada | Parámetros | Si se omite / se hace distinto |
|---|---|---|
| `optimizer.zero_grad(set_to_none=True)` | `set_to_none`: pone `.grad=None` en vez de ceros. | **Si no se llama**, los gradientes se **acumulan** entre batches (sin error, entrenamiento incorrecto). Debe ir **antes** de `backward()` (o al principio de cada iteración). |

| Llamada | Parámetros | Si se omite / se hace distinto |
|---|---|---|
| `loss.backward(retain_graph=False)` | `retain_graph=True` conserva el grafo. | Dos `backward()` seguidos sin `retain_graph` → `RuntimeError: Trying to backward through the graph a second time`. Si ningún parámetro tiene `requires_grad` → `element 0 of tensors does not require grad`. Con loss no escalar → `grad can be implicitly created only for scalar outputs`. |
| `optimizer.step()` | — | Si se llama **sin** `backward()` antes, no actualiza nada (`.grad` es `None`). Se llama **después** de `backward()` y del clipping. |

**`torch.nn.utils.clip_grad_norm_(parameters, max_norm, norm_type=2.0, error_if_nonfinite=False)`**

- `parameters`: `model.parameters()`. `max_norm`: umbral (ej. `0.01`); **obligatorio**.
- La norma se calcula **conjuntamente sobre todos los parámetros** (no por capa); si supera `max_norm`, todos los gradientes se reescalan.
- `norm_type`: `2.0` (euclídea) por defecto; `inf` usa el máximo.
- `error_if_nonfinite=True` lanza `RuntimeError` si la norma es `nan/inf` (por defecto lo deja pasar).
- Va **entre `backward()` y `step()`**; si va antes de `backward()` no hace nada. Devuelve la norma original. (`clip_grad_value_(params, clip_value)` recorta cada gradiente a ±valor.)
- `max_norm` muy pequeño (0.01) frena el aprendizaje.

**`torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma, last_epoch=-1)`**

- `optimizer`: el optimizador ya creado. `gamma`: factor multiplicativo (0.9 → el lr se multiplica por 0.9 cada vez). **Obligatorio**.
- `gamma=1` no cambia el lr; `gamma>1` lo **aumenta**; `gamma<=0` lleva el lr a 0 o a valores sin sentido.
- Se llama a `scheduler.step()` **una vez por época, después de `optimizer.step()`**. Si se llama antes → `UserWarning` y se salta el primer valor.
- Ver el lr actual: `optimizer.param_groups[0]["lr"]`.

---

# 9. Datos

**`TensorDataset(*tensors)`**

Empareja tensores por filas (`X`, `Y`). Todos con el mismo nº de filas, si no → `AssertionError: Size mismatch between tensors`. Sin tensores → error. `ds[i]` devuelve la tupla `(X[i], Y[i])`; `len(ds)` = nº de muestras.

**`DataLoader(dataset, batch_size=1, shuffle=False, sampler=None, num_workers=0, drop_last=False, pin_memory=False)`**

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `dataset` | Un `Dataset` (p. ej. `TensorDataset`). **Obligatorio.** | `TypeError`. | Pasar tensores sueltos → falla al iterar. |
| `batch_size` | Muestras por iteración (16, 64, 128...). | `1` (entrena muestra a muestra, muy lento y ruidoso). | `0` o negativo → `ValueError`. Mayor que el dataset → un único batch con todo (válido). `len(dl) = ceil(N/batch_size)`. |

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `shuffle` | `True` baraja en cada época. | `False`. | Usar `True` en **train**, `False` en val/test. `shuffle=True` junto con `sampler` → `ValueError: sampler option is mutually exclusive with shuffle`. |
| `drop_last` | Descarta el último batch si es incompleto. | `False`. | `True` evita batches de 1 muestra (problema con `BatchNorm`). Si `N < batch_size` con `True` → el loader queda **vacío** (0 batches, división por cero al promediar). |
| `num_workers` | Procesos que cargan datos. | `0` (en el proceso principal). | `>0` puede fallar en Windows/notebooks sin `if __name__ == "__main__"`. Para datos tabulares pequeños no compensa. |
| `pin_memory` | Acelera copias a GPU. | `False`. | Solo útil con GPU. |

**`train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)`**

| Parámetro | Qué es y qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `*arrays` | `X`, `y`… (arrays, tensores, DataFrames) con la misma longitud. Devuelve `X_tr, X_te, y_tr, y_te`. | — | Longitudes distintas → `ValueError: Found input variables with inconsistent numbers of samples`. |
| `test_size` | Float (0–1) = proporción, o entero = nº de muestras. | Si tampoco hay `train_size`: **0.25**. | `1.5` o `0` → `ValueError: test_size=... should be either positive and smaller than the number of samples...`. |
| `train_size` | Igual, para train. | Complemento de `test_size`. | `test_size + train_size > 1` → `ValueError`. |
| `random_state` | Entero → reparto **reproducible**. | `None`: reparto distinto cada vez. | Con la misma semilla y los mismos datos el reparto es idéntico. |
| `shuffle` | Baraja antes de dividir. | `True`. | `False` con `stratify` → `ValueError: Stratified train/test split is not implemented for shuffle=False`. |
| `stratify` | Array de etiquetas (normalmente `y`) para mantener las proporciones por clase. | `None`. | Una clase con **1 sola muestra** → `ValueError: The least populated class in y has only 1 member, which is too few` (por eso `make_loaders` lo desactiva en el segundo split). Con tensores torch funciona, pero conviene pasar NumPy. |

Para hacer 70/15/15: dos splits (primero `test_size=0.3`; luego a la mitad `test_size=0.5`).

**`StandardScaler(copy=True, with_mean=True, with_std=True)`**

Resta la media y divide por la desviación → media 0, desviación 1. - `.fit(X_train)`: calcula media y std **solo con train** (ajustar con todo el dataset = fuga de datos). `X` debe ser 2D `(n_muestras, n_features)`. - `.transform(X)`: aplica el escalado. Con otro nº de columnas → `ValueError: X has 4 features, but StandardScaler is expecting 5`. - `.inverse_transform(X)`: deshace el escalado (para devolver predicciones a unidades reales). - `with_mean=False`: solo divide (no centra). `with_std=False`: solo resta la media. - `fit` con un vector 1D (`y`) → `ValueError: Expected 2D array, got 1D array instead`: usar `y.reshape(-1,1)`. - Una columna constante → std=0; sklearn la deja como 0 sin error.

---

# 10. Regularización

**`nn.Dropout(p=0.5, inplace=False)`**

- `p`: probabilidad de **apagar** cada neurona (float 0–1). Típico 0.1–0.5.
- Si se omite: `0.5`. `p=0` no hace nada; `p=1` anula toda la salida; fuera de [0,1] → `ValueError: dropout probability has to be between 0 and 1`.
- Solo actúa con `model.train()`; en `model.eval()` deja pasar todo. Olvidar `eval()` → predicciones ruidosas (sin error).
- En entrenamiento escala por `1/(1-p)` lo que no apaga, para mantener la media.

**`nn.BatchNorm1d(num_features, eps=1e-5, momentum=0.1, affine=True, track_running_stats=True)`**

| Parámetro | Qué es | Si se omite | Otros valores / errores |
|---|---|---|---|
| `num_features` | Nº de columnas de la entrada = `out_features` de la capa anterior. | Obligatorio. | No coincide → `RuntimeError: running_mean should contain X elements not Y`. |
| `eps` | Pequeño valor para evitar dividir por 0. | `1e-5`. | — |
| `momentum` | Peso de las estadísticas del batch en la media móvil. | `0.1`. | `None` → media acumulada simple. |
| `affine` | Si aprende escala `γ` y desplazamiento `β` (2·`num_features` parámetros). | `True`. | `False`: sin parámetros entrenables. |
| `track_running_stats` | Guarda medias móviles para usarlas en `eval()`. | `True`. | `False`: usa estadísticas del batch también en `eval()`. |

Entrada admitida: `(B,C)` o `(B,C,L)`; otra dimensión → `ValueError: expected 2D or 3D input`. En `train()` con **batch de 1 muestra** → `ValueError: Expected more than 1 value per channel when training`. Orden habitual: `Linear → BatchNorm → ReLU → Dropout`.

### Regularización L1 (a mano)

`loss = loss + l1_lambda * sum(p.abs().sum() for p in model.parameters())` — se suma **antes** de `backward()`. `l1_lambda` (ej. 0.001): grande → pesos a 0; `0` → sin efecto. `torch.linalg.norm(p, 1)` sobre una matriz 2D **no** es la suma de valores absolutos (es la norma-1 matricial: máximo de las sumas por columna); en vectores (bias) sí.

---

# 11. Modos, device y gradientes

| Elemento | Parámetros | Qué hace / errores |
|---|---|---|
| `model.train(mode=True)` | `mode`. | Activa Dropout y BatchNorm de entrenamiento. `model.train(False)` = `eval()`. |
| `model.eval()` | — | Para validar/inferir. No desactiva gradientes (para eso `torch.no_grad()`). |
| `with torch.no_grad():` | — | No construye el grafo → menos memoria, y las salidas ya no tienen `requires_grad`. Si se hace `loss.backward()` dentro → `element 0 of tensors does not require grad`. |

| Elemento | Parámetros | Qué hace / errores |
|---|---|---|
| `@torch.no_grad()` | Decorador. | Igual, aplicado a toda una función (`predict`). |
| `model.to(device)` | `device`: `"cpu"` / `"cuda"`. | Modifica el modelo **in situ**. Los datos (`xb.to(device)`) hay que moverlos aparte y devuelven copia; si no se reasigna, no se mueven. |
| `model.parameters()` | — | Iterador de tensores entrenables (lo que recibe el optimizador). |
| `model.named_parameters()` | — | Pares `(nombre, tensor)`: `"0.weight"`, `"0.bias"`, `"2.weight"`... |
| `p.requires_grad = False` | bool. | Congela ese parámetro. Congelarlo **tras** crear el optimizador no lo saca del optimizador (simplemente no recibe `.grad`). Congelar todo → `backward()` falla. |

## 12. Inicialización

Funciones de `torch.nn.init`. Terminan en `_` = modifican el tensor **in situ**. Se aplican a `m.weight` (2D); sobre un bias 1D, `kaiming/xavier` dan `ValueError: Fan in and fan out can not be computed for tensor with fewer than 2 dimensions`.

| Función | Parámetros | Si se omiten / otros valores |
|---|---|---|
| `kaiming_uniform_(tensor, a=0, mode='fan_in', nonlinearity='leaky_relu')` | `a`: pendiente negativa de la activación; `mode`: `'fan_in'` o `'fan_out'`; `nonlinearity`: `'relu'`, `'leaky_relu'`, `'tanh'`... | Con los defectos vale para ReLU. `mode` inválido → `ValueError: Mode ... not supported`. Recomendado `nonlinearity="relu"`. |
| `xavier_uniform_(tensor, gain=1.0)` | `gain`: factor (p. ej. `nn.init.calculate_gain('tanh')`). | `gain=1`. Adecuado para tanh/sigmoid. |
| `normal_(tensor, mean=0.0, std=1.0)` | Media y desviación. | `std` negativo → `RuntimeError`. |
| `constant_(tensor, val)` | `val` obligatorio. | Todo el tensor = `val`. |
| `zeros_(tensor)` / `ones_(tensor)` | — | Inicializar **todos los pesos** a 0/constante → todas las neuronas iguales (no aprenden distinto). |

- **Por defecto**, `nn.Linear` usa Kaiming uniform (`a=√5`) en pesos y uniforme `±1/√in` en el bias.
- `model.apply(fn)`: ejecuta `fn(módulo)` sobre **cada submódulo** recursivamente. Por eso `fn` filtra con `isinstance(m, nn.Linear)`; sin filtro, `ReLU`/`Sequential` no tienen `.weight` → `AttributeError`. `apply` devuelve el modelo.

## 13. Guardar y cargar

| Llamada | Parámetros | Errores / efectos |
|---|---|---|
| `model.state_dict()` | — | Diccionario nombre→tensor. Devuelve **referencias**: para guardar "el mejor" durante el entrenamiento usar `copy.deepcopy(model.state_dict())`, si no cambia al seguir entrenando. |

| Llamada | Parámetros | Errores / efectos |
|---|---|---|
| `torch.save(obj, f)` | `obj`: normalmente el `state_dict`; `f`: ruta (`.pt`/`.pth`). | Ruta con carpeta inexistente → `FileNotFoundError`. Guardar el modelo entero (`torch.save(model, ...)`) es frágil (depende del código de la clase). |
| `torch.load(f, map_location=None, weights_only=True)` | `f`: ruta. `map_location`: `"cpu"` para cargar en CPU algo guardado en GPU. `weights_only`: solo permite tensores. | Sin `map_location`, un archivo de GPU en una máquina sin GPU → `RuntimeError: Attempting to deserialize object on a CUDA device`. En versiones recientes `weights_only=True` es el defecto; un modelo entero guardado da `UnpicklingError: Weights only load failed`. Archivo inexistente → `FileNotFoundError`. |
| `model.load_state_dict(state_dict, strict=True)` | `strict`: exigir las mismas claves. | `True` y claves distintas → `RuntimeError: Missing key(s)/Unexpected key(s)`. `strict=False` ignora claves que faltan/sobran (devuelve las listas), **pero un tamaño distinto sigue dando error** `size mismatch for 0.weight`. Hay que crear antes el modelo con **la misma arquitectura**; después, `model.eval()`. |

## 14. Métricas (scikit-learn)

Todas reciben `y_true` (reales) e `y_pred` (predichos) como arrays/listas 1D de etiquetas **enteras de clase** (no probabilidades ni logits).

| Función | Parámetros | Si se omiten / otros valores / errores |
|---|---|---|
| `accuracy_score(y_true, y_pred, normalize=True, sample_weight=None)` | `normalize=False` devuelve el nº de aciertos. | Longitudes distintas → `ValueError: inconsistent numbers of samples`. Pasar logits (B,C) → `ValueError: ...multilabel-indicator and multiclass`. |
| `balanced_accuracy_score(y_true, y_pred, adjusted=False)` | `adjusted=True` descuenta el azar. | Media del recall por clase. Con clases ausentes en `y_true` avisa (`y_pred contains classes not in y_true`). |
| `f1_score(y_true, y_pred, labels=None, pos_label=1, average='binary', zero_division='warn')` | `average`: `'binary'`, `'micro'`, `'macro'`, `'weighted'` o `None` (uno por clase). `zero_division`: `0`, `1` o `'warn'`. | **Por defecto `'binary'`**: con más de 2 clases → `ValueError: Target is multiclass but average='binary'` → usar `average="macro"`. `zero_division=0` silencia el warning de clases sin predicciones. |
| `confusion_matrix(y_true, y_pred, labels=None, normalize=None)` | `labels`: orden/lista de clases. `normalize`: `'true'`, `'pred'`, `'all'`. | `labels=None`: solo las clases que aparecen (matriz más pequeña si faltan). `labels=range(C)` fuerza todas (filas = reales, columnas = predichas). Con `normalize='true'` cada fila suma 1 (recall por clase). Una fila sin muestras en la normalización manual divide por 0 → por eso `clip(min=1)`. |
| `ConfusionMatrixDisplay(confusion_matrix, display_labels=None).plot(cmap='viridis')` | Etiquetas de ejes; mapa de color. | Sin `display_labels` usa 0..n-1. `plt.savefig(ruta)` guarda la imagen; sin `plt.show()` o `savefig` en un script no se ve nada. |
| `datasets.load_iris(return_X_y=False, as_frame=False)` | `return_X_y=True` devuelve `(X, y)`. `as_frame=True` da DataFrame. | Por defecto un objeto con `.data` (150×4), `.target` (0,1,2), `.target_names`. |

## 15. pandas y NumPy usados

| Función | Parámetros | Qué pasa |
|---|---|---|
| `pd.read_csv(ruta, sep=',', header='infer', nrows=None, index_col=None)` | `sep`: separador. | Wine usa `;`: sin `sep=";"` queda **una sola columna** y luego `KeyError: 'quality'`. Ruta mal → `FileNotFoundError`. `nrows=5` lee solo 5 filas. |
| `pd.concat(objs, axis=0, ignore_index=False)` | `axis=0` apila filas; `axis=1` columnas. `ignore_index=True` renumera. | Sin `ignore_index` quedan índices repetidos. |
| `df.drop_duplicates(subset=None, keep='first', inplace=False)` | `subset`: columnas a comparar; `keep`: `'first'`, `'last'`, `False` (elimina todos los duplicados). | Devuelve **copia**: hay que reasignar. `inplace=True` modifica y devuelve `None`. |
| `df.reset_index(drop=False)` | `drop=True` descarta el índice antiguo. | Sin `drop=True` se crea una columna `index` extra (que entraría como feature por error). |
| `df.duplicated()` | `.sum()` cuenta duplicadas. | — |
| `df["col"].value_counts().sort_index()` | — | Cuenta por clase, ordenado por valor de la clase. |
| `df.values` / `df.drop(columns=...)` | — | `drop(columns="quality")` quita la columna; nombre inexistente → `KeyError`. |
| `np.bincount(x, minlength=0)` | `x`: enteros ≥ 0. `minlength`: longitud mínima. | Negativos → `ValueError`. Con `minlength=n_classes` asegura contar clases ausentes (0). Si `x` es float → `TypeError`. |
| `np.unique(x)`, `np.sort`, `np.clip(x, a_min, a_max)`, `np.rint(x)` | `rint` redondea al entero más cercano; `clip` limita a un rango. | `rint` devuelve floats: convertir con `astype(int)` si se usan como índices. |

---

## 16. Funciones definidas en `tabular_data.ipynb` y `training_iris.py`

**`make_loaders(X, y, task="regression", val_size=0.15, test_size=0.15, batch_size=128, stratify=None)`**

| Parámetro | Qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `X`, `y` | Arrays NumPy: `X` (N,F), `y` (N,). | Obligatorios. | `y` de clasificación debe ser entero `0..C-1`. |
| `task` | `"regression"` o `"classification"`. | `"regression"`: escala también `y` y la devuelve `float (N,1)`. | `"classification"` → `y` como `long`. Cualquier otra cadena **no da error**: se trata como clasificación. |
| `val_size`, `test_size` | Proporciones (0.15 = 15 %). | 0.15 y 0.15 → 70/15/15. | `val_size+test_size ≥ 1` → `ValueError` de sklearn. |

| Parámetro | Qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `batch_size` | Batch del loader de **train** (val/test usan 1024). | 128. | — |
| `stratify` | Etiquetas (p. ej. `y_cls`) o `None`. | `None`. | Con una clase de <2 muestras en val+test, el segundo split se hace sin estratificar. |

Devuelve: `dl_tr, dl_va, dl_te, sx, sy, (y_tr, y_va, y_te)`; `sy=None` en clasificación.

**`fit(model, dl_tr, dl_va, loss_fn, epochs=100, lr=1e-3, weight_decay=0.0, verbose=True)`**

| Parámetro | Qué meter | Si se omite | Otros valores / errores |
|---|---|---|---|
| `model` | `nn.Module` (se mueve a `device`). | Obligatorio. | — |
| `dl_tr`, `dl_va` | DataLoaders de train y validación. | Obligatorios. | `dl_va` vacío → división por cero (`vn=0`). |
| `loss_fn` | Objeto loss (`nn.MSELoss()`, `nn.CrossEntropyLoss(weight=...)`). | Obligatorio. | Pasar la clase sin `()` → `TypeError`. Con `weight`, el tensor debe estar en `device`. |
| `epochs` | Nº de pasadas completas por el train. | 100. | `0` → no entrena y `load_state_dict(None)` falla con `TypeError`. |
| `lr` | Tasa de aprendizaje de **Adam**. | `1e-3`. | Ver §8. |
| `weight_decay` | L2. | `0.0`. | — |
| `verbose` | `True` imprime ~10 líneas (`epochs//10`). | `True`. | `False` no imprime. |

Guarda el mejor `state_dict` (por menor val loss) y lo restaura al final; devuelve `{"train": [...], "val": [...]}`.

**`predict(model, dl)`**

Devuelve los logits/predicciones de todo el loader concatenados (CPU). En clasificación `.argmax(1)`; en regresión `sy.inverse_transform(pred.numpy())` para volver a las unidades reales.

**`run_ccpp(target, epochs=100, hidden=(64,64,32))`**

- `target`: **nombre de columna** a predecir (`"PE"` o `"AT"`); el resto de columnas son las features. Nombre inexistente → `KeyError`/lista vacía de features.
- `epochs`, `hidden`: como en `fit` y `MLP`.

**`run_wine_cls(weighted, epochs=150, seed=SEED)`**

- `weighted`: `True` usa `weight = N/(C·cnt)`; `False` loss sin pesos. **Obligatorio**.
- `epochs`: 150 por defecto. `seed`: semilla de PyTorch (se usa en el bucle de 5 semillas).
- Si el split estratificado deja una clase sin muestras en train, `np.bincount(..., minlength=C)` da un 0 y `N/(C·0) = inf`: pesos infinitos y loss `nan`.

**`visualize_true_vs_pred_regression(y_true, y_pred, target_name="PE")`**

`y_true`, `y_pred`: arrays 1D de igual longitud en **unidades reales** (no estandarizadas). `target_name`: texto de los ejes. Tamaños distintos → `ValueError` en `scatter`.

**`plot_history(hist, title="")`**

`hist`: el diccionario que devuelve `fit`. `title`: título. Otra estructura → `KeyError: 'train'`.

**`calculate_confusion_matrix(test_dataloader, model, file=None)` (`training_iris.py`)**

- `test_dataloader`, `model`: obligatorios.
- `file`: ruta (`"cm.png"`). Si se omite (`None`) solo devuelve la matriz y **no** guarda imagen.
- Falta `model.eval()` (no afecta mientras no haya Dropout/BatchNorm).

### Early stopping a mano

| Variable | Significado | Valor y efecto |
|---|---|---|
| `patience` | Épocas seguidas sin mejora que se toleran. | Bajo (1–3): para pronto, riesgo de quedarse corto. Alto: casi no corta. `0` corta a la primera época sin mejora. |
| `min_delta` | Mejora mínima para contar como mejora. | `0`: cualquier mejora cuenta. Grande (como el `10` del ejemplo): casi nunca cuenta como mejora → corta enseguida. |
| `counter` | Épocas sin mejora acumuladas. | Se pone a 0 cuando mejora la val loss. |
| `min_validation_loss` | Mejor val loss vista. | Se inicia en `float('inf')`. |

Lo habitual es `if val < best - min_delta: guardar mejor y counter=0; else: counter+=1; if counter>=patience: break`. El bloque del notebook falla con `NameError` porque `train_one_epoch`, `validate_one_epoch` y `train_loader` no están definidos.

---

## 17. Chuleta de errores

| Mensaje | Causa |
|---|---|
| `mat1 and mat2 must have the same dtype` | Entrada Long/float64 → `.float()` |
| `mat1 and mat2 shapes cannot be multiplied` | `in_features` ≠ última dim |
| `Target N is out of bounds` | Etiquetas fuera de `0..C-1` |
| `Can't call numpy() on Tensor that requires grad` | Falta `.detach()` |

| Mensaje | Causa |
|---|---|
| `Expected all tensors to be on the same device` | Mezcla cpu/cuda |
| `Expected more than 1 value per channel` | BatchNorm con batch de 1 en train |
| `Trying to backward through the graph a second time` | Dos `backward()` |
| `Invalid learning rate / momentum / weight_decay` | Valor negativo |
| `Size mismatch between tensors` | `X` e `Y` con distinta longitud |
| `Expected 2D array, got 1D array` | Falta `reshape(-1,1)` |
| `Target is multiclass but average='binary'` | Falta `average="macro"` en `f1_score` |
| `size mismatch for ...weight` | Arquitectura distinta al cargar |
| **Sin error pero mal** | `zero_grad` olvidado, `Softmax(dim=0)`, `(B)` vs `(B,1)` en MSE, `eval()` olvidado, escalar con todo el dataset, `argmax` sin `dim`, `hidden=(64)` mal escrito, softmax antes de `CrossEntropyLoss` |